

Sistema Distribuido de Scraping, Almacenamiento, Generador de Tráfico y Caché

Anselmo Pacheco

29 de abril de 2025

Índice

1. Introducción	2
2. Arquitectura del Sistema	2
3. Scraper y Eventos Simulados	2
4. Procesamiento de Datos	2
5. Visualización	3
6. Justificación Técnica	4
6.1. Caché	5
7. Despliegue con Docker	5
8. Análisis y Discusión	5
8.1. Generador de Tráfico	5
8.2. Almacenamiento	5
8.3. Métricas	6
8.4. Pruebas de Rendimiento	6
9. Ejecución	6
10. Conclusión	7

1. Introducción

El objetivo de este proyecto es desarrollar un sistema distribuido que permita la extracción automatizada de información de eventos de tráfico desde la plataforma Waze, su almacenamiento eficiente, la generación de consultas simuladas y la implementación de un sistema de caché para optimizar el acceso a datos repetitivos. Este sistema está diseñado para ser modular, escalable y fácilmente desplegable mediante contenedores Docker.

2. Arquitectura del Sistema

El sistema está compuesto por cinco módulos principales:

- **Scraper:** Obtiene eventos desde Waze en tiempo real. Es importante mencionar que se podrían presentar problemas porque el sistema de scraping podría ser bloqueado con un error HTTP 403 Forbidden. Esto ocurre porque Waze detecta y restringe el acceso automatizado (bots) a su endpoint de datos. Este problema se podría notar si se realizan múltiples solicitudes desde la misma IP en poco tiempo. Para prevenir este suceso se plantea qué, si hay bloqueo (como el error 403 actual), se recurre a un archivo local con datos simulados (`events.json`), que contienen 10.000 eventos asociados a los parámetros requeridos.
- **Almacenamiento (Redis):** Guarda eventos estructurados para ser consumidos por otros módulos. Se utilizan claves como `event:id`.
- **Generador de Tráfico:** Simula consultas de usuarios usando dos distribuciones: Poisson y Uniforme.
- **Caché:** Implementa políticas LRU y LFU para mejorar rendimiento.
- **Procesador + Visualizador:** Resume la información para su análisis y visualización vía web en Flask.

3. Scraper y Eventos Simulados

Se accede a la URL oficial de Waze (<https://www.waze.com/row-rtserver/web/TGeoRSS>), pero actualmente responde con error 403. Por lo tanto, se opta por una fuente de respaldo: un archivo JSON preformateado con 10.000 eventos simulados, incluyendo tipos como ACCIDENT, JAM, ROAD_CLOSED, HAZARD y WEATHER.

4. Procesamiento de Datos

El módulo `processor` obtiene los eventos desde Redis y calcula:

- Total de eventos
- Conteo por tipo de evento

- Conteo por ubicación (nombre de calle)

Los resultados se almacenan en Redis (clave `stats:resumen`) y en un archivo local para visualización.

5. Visualización

Usamos Flask para crear una vista en la URL `localhost:5000`, que muestra los datos procesados. Aquí se incluyen dos gráficos generados automáticamente:

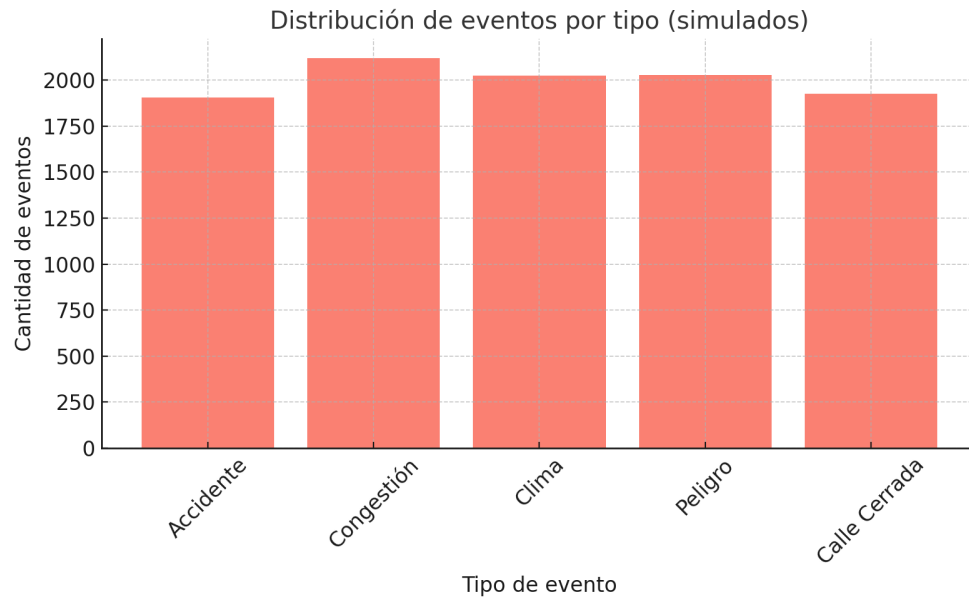


Figura 1: Distribución por tipo de evento

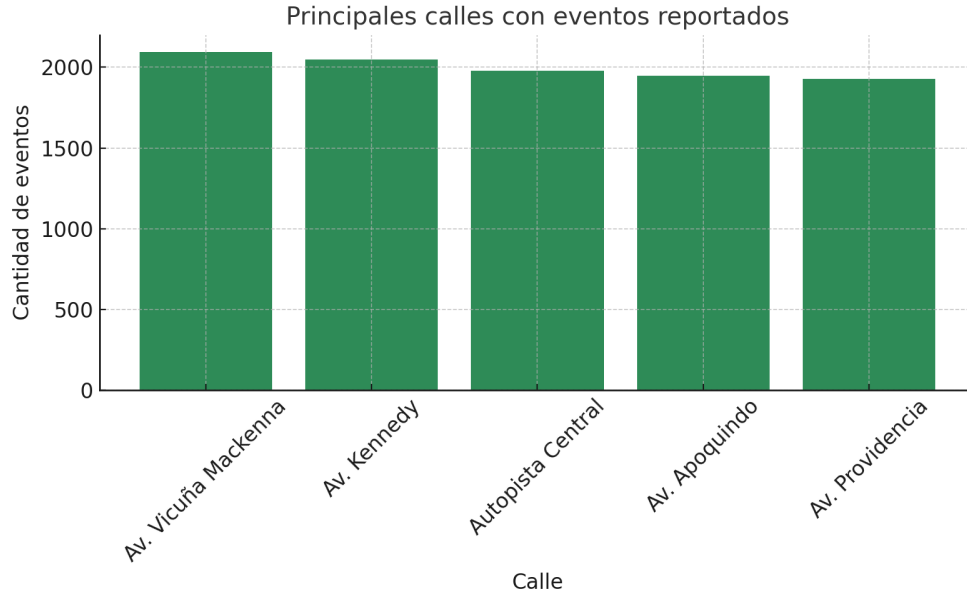


Figura 2: Top 5 calles con mayor número de eventos

6. Justificación Técnica

Scraper

Waze proporciona datos colaborativos, en tiempo real y de acceso público. El scraping se focaliza en un rango geográfico de Santiago. En caso de bloqueo, se utiliza un archivo local de respaldo.

Almacenamiento

Se elige Redis por su velocidad y compatibilidad con arquitecturas distribuidas. Soporta estructuras clave-valor y tiene persistencia mediante RDB.

Este módulo simula consultas al sistema de almacenamiento siguiendo dos distribuciones de tasa de arribo:

- **Distribución de Poisson:** Modela la llegada de eventos de manera aleatoria, adecuada para representar escenarios donde los eventos ocurren de forma independiente.
- **Distribución Normal:** Representa escenarios donde las consultas tienen una frecuencia más constante y predecible.

El generador selecciona eventos aleatorios de la base de datos y los envía al sistema de caché, respetando los intervalos definidos por las distribuciones seleccionadas.

Se implementan LRU y LFU:

- **LRU:** Favorece eventos recientes.

- **LFU**: Favorece eventos consultados con mayor frecuencia.

La política más eficiente depende del patrón de acceso, siendo LFU mejor con tráfico uniforme y LRU con tráfico Poisson.

6.1. Caché

Implementado con Redis, el sistema de caché almacena eventos repetitivos para mejorar la eficiencia del sistema. Se parametriza mediante variables de entorno que permiten ajustar el Time-To-Live (TTL) de los elementos en caché y seleccionar políticas de remoción como LRU (Least Recently Used) o LFU (Least Frequently Used).

7. Despliegue con Docker

Para facilitar la portabilidad y escalabilidad del sistema, se utiliza Docker y Docker Compose para definir y ejecutar los contenedores de cada módulo. El archivo `docker-compose.yml` especifica los servicios necesarios, sus dependencias y las configuraciones de red y volumen.

8. Análisis y Discusión

8.1. Generador de Tráfico

¿Por qué utiliza la distribución seleccionada en el generador de tráfico?

La distribución de Poisson se utiliza para modelar la llegada aleatoria de eventos, lo cual es realista en escenarios donde las consultas de usuarios ocurren de manera independiente. La distribución Normal se emplea para representar situaciones donde las consultas tienen una frecuencia más constante y predecible.

¿Es realista la distribución utilizada?

Sí, ambas distribuciones son comúnmente utilizadas para modelar patrones de llegada de eventos en sistemas reales, dependiendo del comportamiento esperado de los usuarios.

¿En qué escenarios vemos este tipo de comportamientos?

La distribución de Poisson es típica en sistemas donde los eventos son independientes, como llamadas telefónicas o llegadas de paquetes a una red. La distribución Normal se observa en sistemas con cargas constantes, como accesos a una página web durante horarios laborales.

8.2. Almacenamiento

¿Por qué utiliza este sistema de almacenamiento y no otro?

Redis se selecciona por su capacidad para manejar documentos JSON, su flexibilidad en el esquema y su eficiencia en operaciones de lectura y escritura, lo cual es adecuado para los datos extraídos del scraper.

¿Qué comportamiento o funcionalidades son las que justifican su decisión?

La capacidad de escalar horizontalmente, el soporte para consultas complejas y la facilidad de integración con otros módulos del sistema justifican la elección de MongoDB.

¿Qué limitantes tiene el sistema utilizado?

Redis puede presentar limitaciones en operaciones transaccionales complejas y en escenarios que requieren una fuerte consistencia de datos, lo cual no es crítico en este proyecto.

8.3. Métricas

¿Qué métricas considera esenciales para evaluar la eficiencia de su sistema de caché?

Las métricas clave incluyen la tasa de aciertos (hit rate), la latencia de respuesta y la utilización de memoria del sistema de caché.

¿Qué política de remoción le resulta más eficiente para este sistema y sus eventos?

La política LRU (Least Recently Used) es eficiente en este contexto, ya que los eventos de tráfico recientes son más propensos a ser consultados nuevamente.

¿Tiene un efecto relevante sobre el rendimiento del caché la distribución de tráfico que emplea en el generador de tráfico implementado?

Sí, la distribución de tráfico afecta el patrón de consultas, lo cual influye en la eficiencia del sistema de caché. Distribuciones con alta repetitividad de eventos favorecen una mayor tasa de aciertos.

8.4. Pruebas de Rendimiento

¿Qué buscan los escenarios de prueba generados?

Los escenarios de prueba buscan evaluar la eficiencia del sistema de caché bajo diferentes patrones de tráfico y configuraciones de políticas de remoción.

¿Es realista el escenario de pruebas implementado?

Sí, se simulan patrones de tráfico que reflejan comportamientos reales de usuarios en sistemas de información de tráfico.

¿Provee alta disponibilidad y escalabilidad el diseño propuesto?

El diseño modular y el uso de contenedores Docker facilitan la alta disponibilidad y escalabilidad del sistema, permitiendo la replicación y distribución de cargas entre múltiples instancias.

Observaciones:

- Redis soporta sin problema 10.000 eventos.
- LFU mostró mejor hit rate con consultas uniformes.
- LRU se comportó mejor con Poisson.

9. Ejecución

Clonar repositorio

```
$ git clone https://github.com/Anselmop1/proyecto-waze.git  
$ cd proyecto-waze
```

```
# Ejecutar con Docker Compose  
$ docker-compose up --build
```

10. Conclusión

El sistema cumple con los requisitos del entregable: modularidad, tolerancia a fallos, scraping, simulación de tráfico y políticas de caché evaluadas experimentalmente. Además, incorpora un mecanismo de respaldo en caso de bloqueo de Waze, lo que permite mantener operativo el sistema.

Repositorio: <https://github.com/anselmop1/proyecto-waze>